



UNIVERSIDAD DE DAGUPAN

SCHOOL OF INFORMATION TECHNOLOGY EDUCATION

ITP10 | EVENT-DRIVEN PROGRAMMING

MIDTERM | LABORATORY ACTIVITY 3

Name : Samson, Laurence D.
Year, Course & blk : 31-ITE-01
Subject : ITP 10 - Event Driven Programming
Date : October 03, 2027

SCORE:

Laboratory Activity #3: Student Registration Form with Input Validation

Objective: At the end of this activity, students should be able to create an HTML form with appropriate controls and labels, use JavaScript events to validate user input, and apply regular expressions to check input formats. They should also be able to display clear, accessible error and success messages and safely display user input using `textContent`.

Scenario

Your university needs a registration form for an upcoming IT seminar. Develop a form that checks student information before displaying a registration summary.

Part A — Create the Registration Form

Create these files:

- `index.html`
- `css/style.css`
- `js/app.js`

Include the following fields and controls:

Field or Control	Requirement
Full name	Text input; required id="fullName"
Student number	Text input; required id="studentNumber"
Email address	Email input; required id="email"
Mobile number	Telephone input; required id="mobileNumber"
Password	Password input; required id="password"
Confirm password	Password input; required id="confirmPassword"
Course	Dropdown with BSIT and BSCS options; required id="course"; option values must be BSIT and BSCS
Terms agreement	Checkbox; required id="terms"
Register	Submit button; required id="registerBtn"
Reset	Reset button; required id="resetBtn"

Every field must have a visible label. The course dropdown must begin with an unselected placeholder such as “**Select your course.**” The form itself must use `id="registrationForm"` so the automated browser tests can submit it consistently.

Part B — Apply Validation Rules

Use HTML attributes and JavaScript checks where appropriate. Use regular expressions for the student number, email, mobile number, and password.

Field	Validation Rules
Full name	Required; at least two characters after trimming; reject input containing only spaces.
Student number	Required; follow the format 24-1234-123: two digits, a hyphen, four digits, a hyphen, and three digits.
Email address	Required; use the module's simplified email pattern: text before @, a domain containing a dot, and text after that dot; no whitespace.
Mobile number	Required; accept 09 followed by nine digits or +639 followed by nine digits; no spaces or hyphens.
Password	Required; at least eight characters, one uppercase letter, one digit, and one of @, \$, or !; no whitespace.
Confirm password	Required; must match the password exactly.
Course	Must select BSIT or BSCS.
Terms agreement	Must be checked.

Important: Trim leading and trailing spaces from the name, student number, email, and mobile number before checking them. Do not trim passwords or remove spaces from the middle of an entry.

Autograder Compatibility Requirements

To make the activity compatible with the Programming Classroom function and browser tests, follow these exact names and rules:

- Create a pure function named `isValidStudentNumber(value)`. It must return true only when the trimmed value matches the required student-number format; otherwise return false.
- Create a pure function named `isValidPassword(value)`. It must return true only when the password satisfies all password rules; otherwise return false.
- The two validation functions must not read from document, window, or any HTML element. They must work only from the argument passed to them.
- Keep `js/app.js` safe to load in Node for the function tests. Do not access document or window at file load time unless the access is guarded with `typeof document !== "undefined"`.
- When CommonJS is available, export the two functions using `module.exports` so the autograder can call them.
- Use `textContent` when writing user-entered values into the registration summary. Do not use `innerHTML` for user-entered values.

Required feedback and summary element IDs

Purpose	Required ID
Full-name error	<code>fullNameError</code>
Student-number error	<code>studentNumberError</code>
Email error	<code>emailError</code>
Mobile-number error	<code>mobileNumberError</code>
Password error	<code>passwordError</code>
Confirm-password error	<code>confirmPasswordError</code>
Course error	<code>courseError</code>
Terms error	<code>termsError</code>
Live password feedback	<code>passwordFeedback</code>
Success message	<code>successMessage</code>

Registration summary container	registrationSummary
Summary full name	summaryName
Summary student number	summaryStudentNumber
Summary email	summaryEmail
Summary mobile number	summaryMobileNumber
Summary course	summaryCourse

Initial-state requirement: `successMessage` and `registrationSummary` must be empty or hidden before a successful registration. Reset must return them to that initial state.

Part C — Add Event Handling and Feedback

Implement the following behavior:

- On form submission**
 - Use the form's `submit` event.
 - Call `event.preventDefault()` because this activity processes the form locally.
 - Check every field and show all relevant errors.
 - Display a success message only when all fields pass validation.
- While entering a password**
 - Use the `input` event.
 - Show feedback explaining whether the password meets the stated rules.
- After leaving the full-name field**
 - Use the `blur` event.
 - Show an error if the trimmed name is empty or too short.
- When changing the course or terms agreement**
 - Use the `change` event.
 - Update the corresponding feedback.
- When resetting the form**
 - Restore the controls to their initial values.
 - Clear error messages, password feedback, the success message, and the registration summary.

Display errors beside or below the relevant field. Use specific messages, such as:

“Enter a student number in the format 24-1234-123.”

When the user corrects an entry and it is checked again, remove its error message.

Part D — Display the Registration Summary

After successful validation, display: Use the required summary IDs listed in the Autograder Compatibility Requirements.

- Full name
- Student number
- Email address
- Mobile number
- Course

Show this message:

Registration details validated successfully!

Use `textContent` to display user-entered information. Do not display either password.

This activity requires no database or server connection. The message confirms that the browser checks passed.

Part E — Accessibility and Testing

Accessibility requirements

- Connect each label to its control using matching `for` and `id` values.
- Use descriptive error text alongside any error colors.

- Set `aria-invalid="true"` on fields that fail validation; clear or change it to `false` when corrected.
- Connect fields to their error messages using `aria-describedby`.
- Ensure the form can be operated using the keyboard.

Automated testing note

The Programming Classroom autograder will check required files, source-code structure, the two pure validation functions, invalid submission behavior, successful registration, and reset behavior. Keep the required file paths, IDs, option values, and function names exactly as written in this activity.

Required test cases

Test	Expected Result
Submit all fields empty	Show required-field errors; no success summary.
Enter a full name containing only spaces	Reject the name.
Enter 24-1234-12 as the student number	Reject the incorrect format.
Enter student@email as the email	Reject it under the stated simplified rule.
Enter 0917-123-4567 as the mobile number	Reject the hyphenated format.
Enter student123 as the password	Reject it for missing an uppercase letter and required symbol.
Enter different passwords	Show a password-mismatch error.
Leave the course unselected or terms unchecked	Prevent successful registration.
Complete all fields correctly	Show the success message and summary.
Click Reset after successful registration	Restore the initial form and clear all feedback and output.

Submission Requirements

- Create a personal GitHub repository for the activity. The repository root must contain `index.html`, `css/style.css`, and `js/app.js`.
- Recommended repository name: Lastname-Firstname-Module6-Lab.
- Include a screenshots folder in the repository containing:
 - The form with validation errors.

Student Registration Form

Full Name

A

Full name must be at least two characters.

Student Number

24-1234-12

Enter a student number in the format 24-1234-123.

Email Address

student@email

Enter a valid email address with a domain and extension.

Mobile Number

0917-123-4567

Enter a mobile number starting with 09 or +639 followed by 9 digits, no spaces or hyphens.

Password

Password must be 8+ characters, include 1 uppercase, 1 digit, 1 special (@!\$), no spaces.

Confirm Password

Passwords do not match.

Course

Select your course

Please select a valid course.

☐ I agree to the terms and conditions. You must agree to the terms and conditions.

- A successful registration summary.

The screenshot shows a web browser window with the title "Student Registration Form". The address bar displays the file path "C:/Users/laure/Downloads/Samson-Laurence-Module6-Lab/index.html". The form contains the following fields and elements:

- Password:** A text input field with masked characters (dots).
- Confirm Password:** A text input field with masked characters (dots).
- Course:** A dropdown menu with "BSIT" selected.
- Agreement:** A checkbox labeled "I agree to the terms and conditions" which is checked.
- Buttons:** A green "Register" button and a red "Reset" button.
- Message:** A green banner stating "Registration details validated successfully!".
- Registration Summary:** A light purple box containing the following details:
 - Full Name:** Juan Dela Cruz
 - Student Number:** 24-1234-123
 - Email:** juan@example.com
 - Mobile Number:** 09171234567
 - Course:** BSIT

- Ensure the activity works by opening index.html in a browser, then push the final version to GitHub.
- Use fictional information and passwords. In GradeVision Programming Classroom, connect the repository and submit the final commit SHA for grading.

Grading Rubric per Part and Function

Part	Function or Requirement	Points
A — Form Structure	Includes all eight required fields and Register and Reset buttons.	10
	Uses appropriate input types, connected labels, and working CSS/JavaScript file links.	5
B — Input Validation	Validates full name: required, trimmed, and at least two characters.	4
	Validates student number using the required format.	4
	Validates email using the specified simplified format.	4
	Validates both accepted Philippine mobile number formats.	4
	Enforces all stated password requirements.	6
	Checks that confirm password matches exactly.	3
	Requires a valid course selection.	2
	Requires the terms agreement to be checked.	3
C — Events and Feedback	Handles submission, prevents page reload, checks all fields, and blocks success when input is invalid.	6
	Uses <code>input</code> for password feedback.	3
	Uses <code>blur</code> for full-name validation.	3
	Uses <code>change</code> for course and terms feedback.	3
	Reset restores initial values and clears all messages and output.	5
D — Registration Summary	Displays all five required details only after successful validation.	6
	Uses <code>textContent</code> , excludes passwords, and displays the success message.	4
E — Accessibility and Testing	Displays specific field-level errors and clears them when corrected and rechecked.	5
	Uses <code>aria-invalid</code> and <code>aria-describedby</code> correctly.	5
	Supports keyboard operation and communicates errors through text alongside colors.	5
	Passes the required test cases.	7
	Submits a complete GitHub repository with the required files/screenshots and the final commit SHA.	3
Total		100

Award full points when a function meets all requirements, partial points when it works incompletely, and zero when it is missing or does not work. Automated test results may be used as evidence for the matching rubric items; requirements not covered by the automated tests remain subject to faculty review.

Time Deduction

Time is measured from the official activity start to the recorded submission time.

Submission Time	Deduction
Within 40 minutes	0 points
More than 40 minutes, up to 60 minutes	–10 points
More than 60 minutes, up to 90 minutes	–20 points
More than 90 minutes, up to 120 minutes	–30 points
More than 120 minutes	–40 points

Final Score = Rubric Score – Time Deduction

Apply only one time deduction based on the submission time.

PREPARED BY:

ROCHELLE S. LANTO, MIT
SITE, FACULTY
JELSON V. LANTO, MIT
SITE, FACULTY